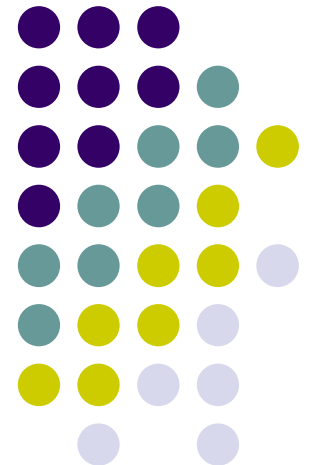


Software Architecture Case Study

Air Traffic Control -
Designing for High Availability





Air Traffic Control (ATC)

- Air Traffic Control (ATC) Readings Chapter 6
- The problem is to control a very large number of aircraft from take-off to landing.
- Problem features:
 - Hard real time – no tolerance for missing deadlines
 - Ultra High availability
 - Safety critical
 - Highly distributed

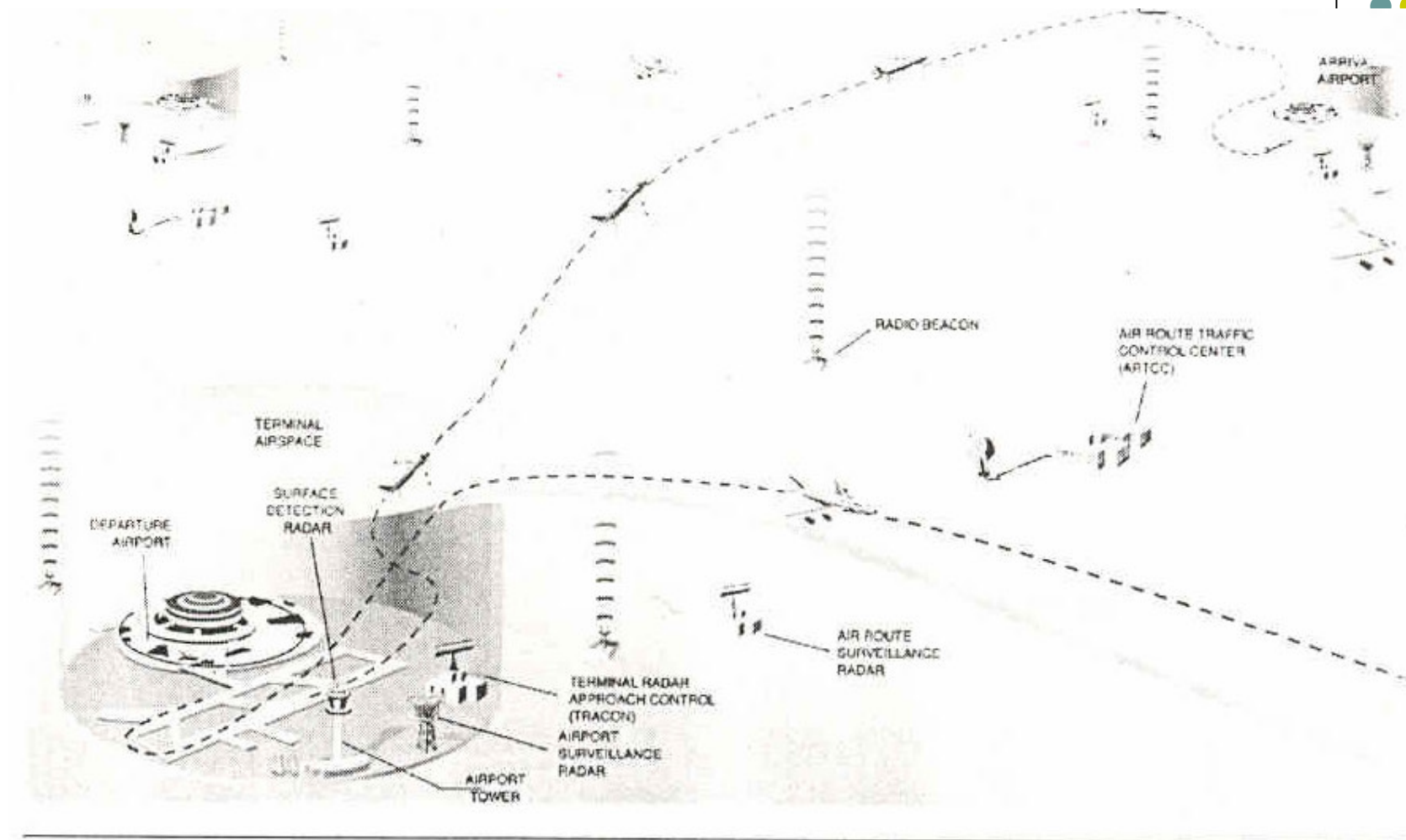
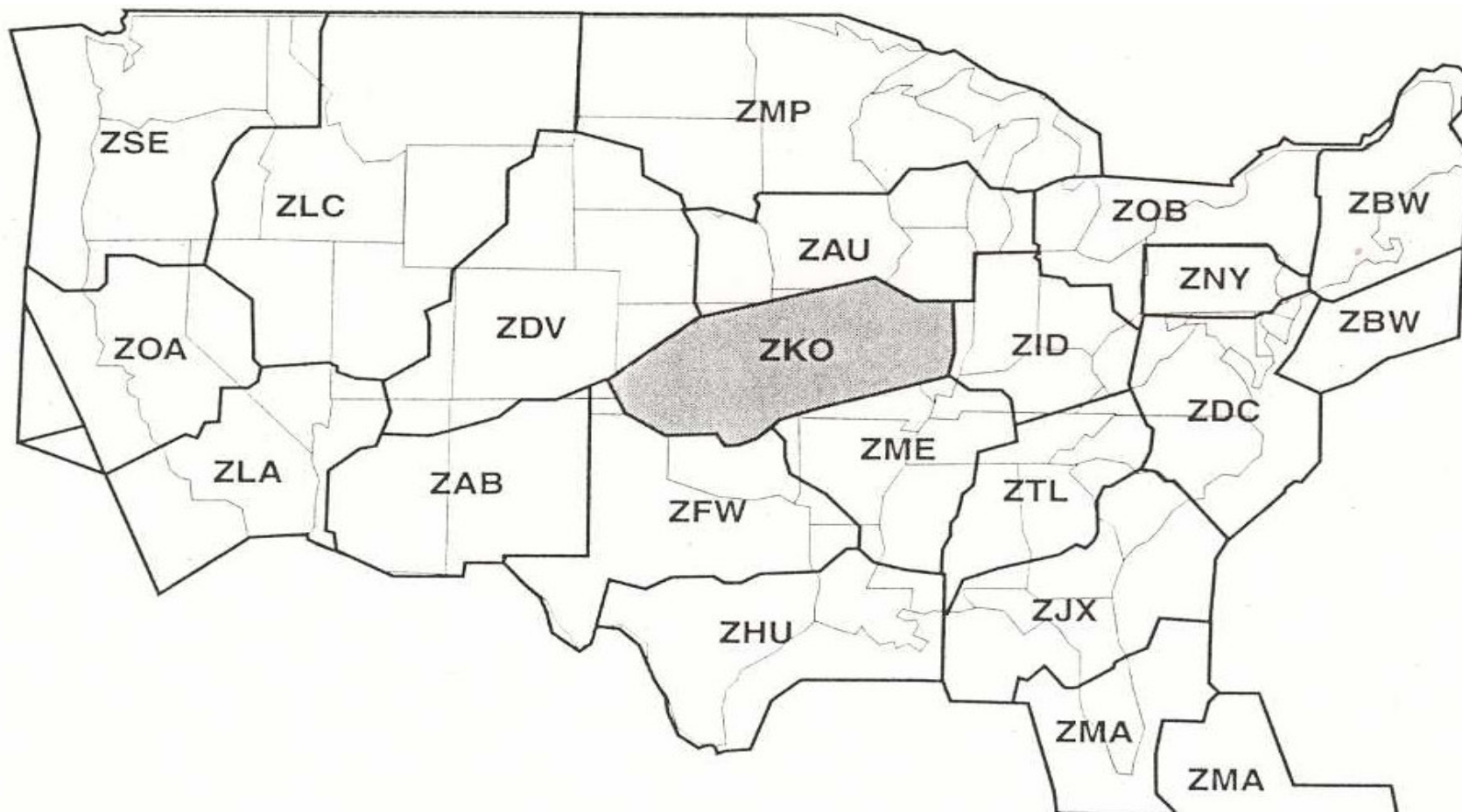


FIGURE 6.1 Flying from point A to point B in the U.S. air traffic control system. Courtesy of Ian Worpole/*Scientific American*, 1994.



En Route Zones in US





Flight Monitoring

- Flight from Key West to DC
 - Key west ground control (to taxi to runway)
 - Key West Tower (take off till leaving airport airspace)
 - ZMA enroute zone center
 - ZJX enroute zone center
 - ZTL enroute zone center
 - ZDC enroute zone center
 - DC Tower (arrival airport)
 - ground-control (to taxi again)
- Advanced Automation System (AAS) Components
 - Ground Control
 - Airport Tower
 - En Route Centers – Initial Sector Suite System (ISSS)
- This study will focus on ISSS only.

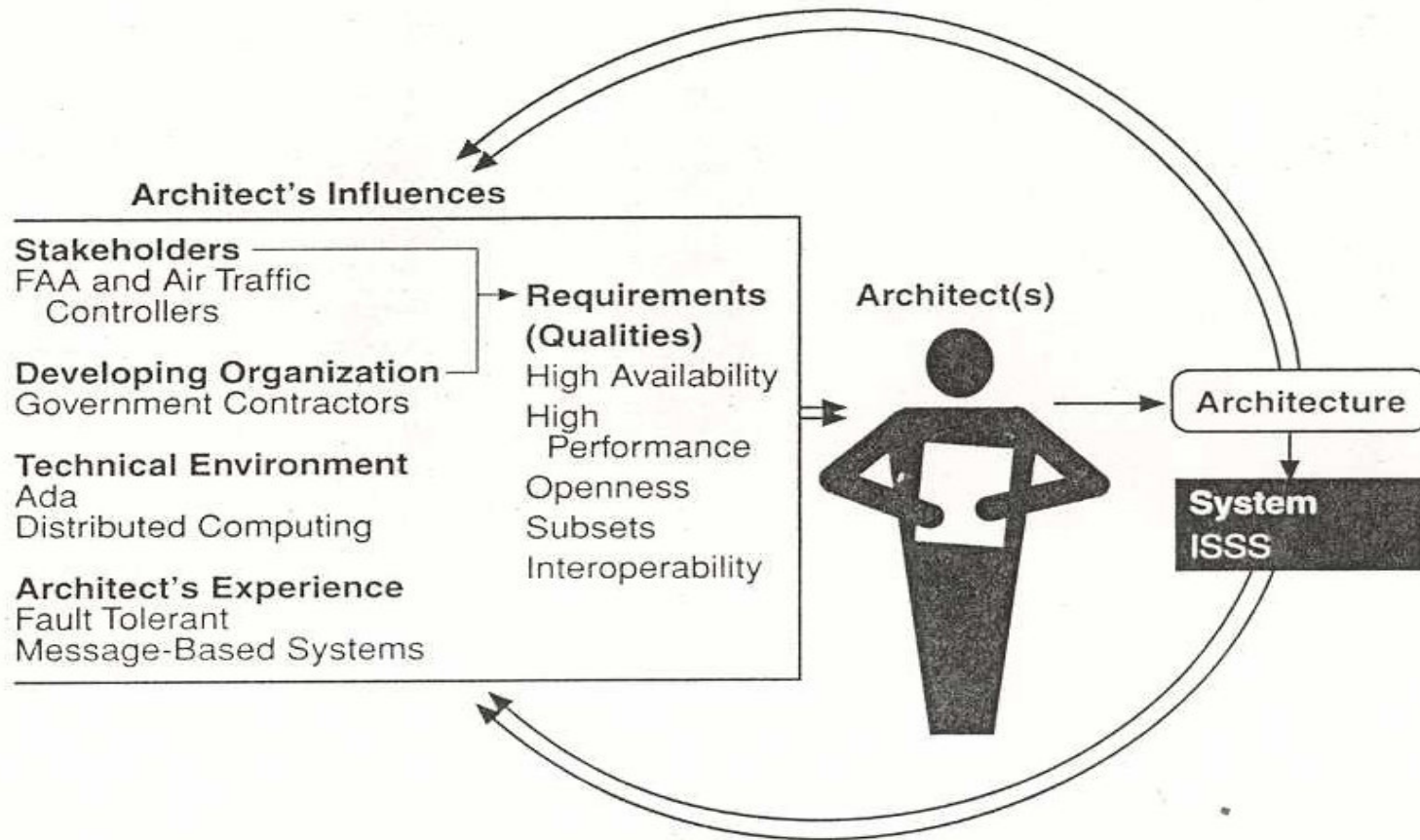


ISSS Influences

- ISSS was only one part of AAS
 - Other components: Ground Control, Airport Tower
- Notes on Design of ISSS
 - Many components in common
 - Interfaces to: radio systems, flight-plan DB, each other
 - Common quality requirements for availability, reliability ...
 - So ISSS was influenced by requirements for all of AAS
- History
 - ISSS real system, designed, most of code developed
 - Not deployed, scaled back to more economical, more staged solution (budget cuts)
 - Outside Audit – the architecture and design were analyzed by an independent audit team that judged “satisfies requirements.”
 - The system deployed borrowed heavily from ISSS
 - <http://home.columbus.rr.com/lusch/blharris.html>



ABC of the Air Traffic Control System





Requirements and Quality Attributes

- ATC system is highly visible with enormous commercial, governmental and public interest
- Great potential for loss of life and costly property.
- Thus the two most important quality attributes were:
 1. Ultrahigh availability
 - Essential that “unavailability” limited to very short periods
 - Availability requirement .99999 → unavailable less than 5 minutes in a year;
 - however short periods (< 10 sec) did not count
 2. High performance
 - Handle up to 2440 aircraft effectively and efficiently



Other Requirements and Quality Attributes

1. Ultrahigh availability
2. High performance
3. Openness- meaning the system needs to be able to incorporate commercially developed components
4. Ability to field subsets
5. Modifiability – modifications to functionality and to handle upgrades in hardware and software
6. Interoperability – the ability to operate with and interface a wide range of external systems



Stakeholders

- FAA
- Controllers – could reject this system if it was not to their liking even if it met all functional requirements
- Usability attribute?
- Actually handled by taking great care with requirements and design (thus slowing the process)



Sector Suites

- Sector Suites – a suite of air-traffic controllers each with their own console that collectively handle all the aircraft in the sector
- Sectors could be defined differently at each center
 - Could be done physically
 - Could be done to balance the load
 - Less densely traveled sectors could be made larger
- Planes are passed off from
 - Departure airport → enroute zone center → ... → arrival airport
 - Also within zone: sector → sector → ... → sector before passing to the next center



ISSS Design

- ISSS requires flexibility in number of control stations per sector (1 to 4)
- At least two controllers per sector:
 1. Radar controller
 - Monitors radar
 - Communicates with aircraft
 - Responsible for maintaining separation of aircraft
 2. Data controller
 - Retrieves flight plans etc.
 - Supplies radar controller with “intentions” of aircraft



ISSS Implementation Metrics

- The system contains about 1 million lines of Ada code
- Designed to support up to 210 consoles per en route center
- Each console was a workstation with IBM RS/6000 processor
- Requirements to handle from 400 to 2440 aircraft simultaneously
- There may be from 16 to 40 radar units to support a center
- A center may have from 60 to 90 control positions



ISSS Functionality Summary

- ISSS must
- Acquire radar targets reports from existing ATC system, the Host Computer System (henceforth “Host”)
- Convert radar reports for display and broadcast to all consoles (consoles can switch areas that are displayed)
- Handle conflict alerts (potential collisions)
- Interface with Host for input and to retrieve flight plans
- Provide extensive monitoring of the system itself to allow dynamic reconfiguration
- Provide recording capability for later playback
- Provide nice GUI
- Provide reduced backup capability in the event of the failure of the Host, the primary network, the primary radar sensors

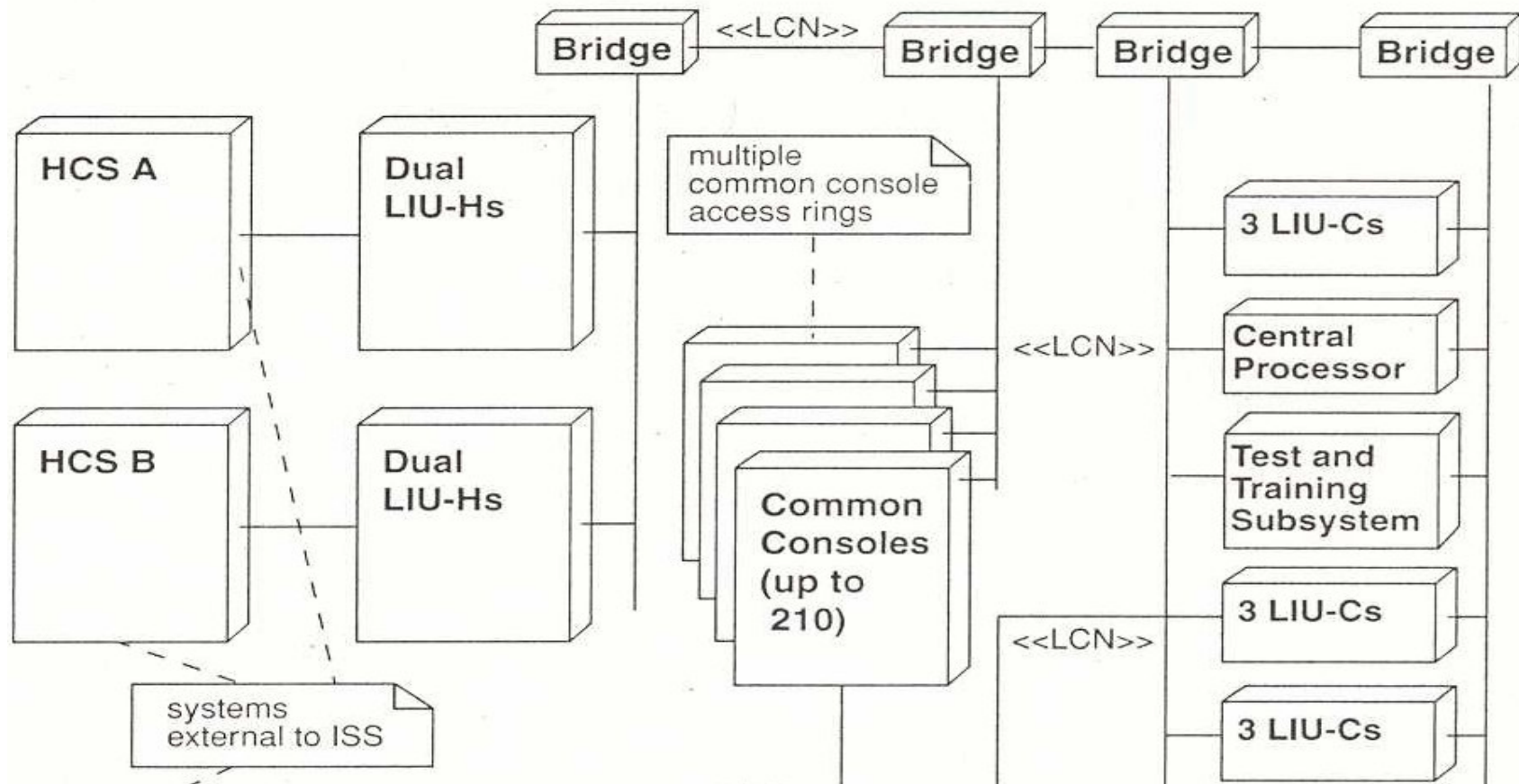


ISSS Architecture

- Remember or two primary and additional quality attributes?
- Which one would you guess had the most influence on architectural decisions?
- Views
 1. Physical View
 2. Module decomposition view
 3. Process View
 4. Client-Server View
 5. Code View
 6. Layered View
 7. Fault Tolerance View

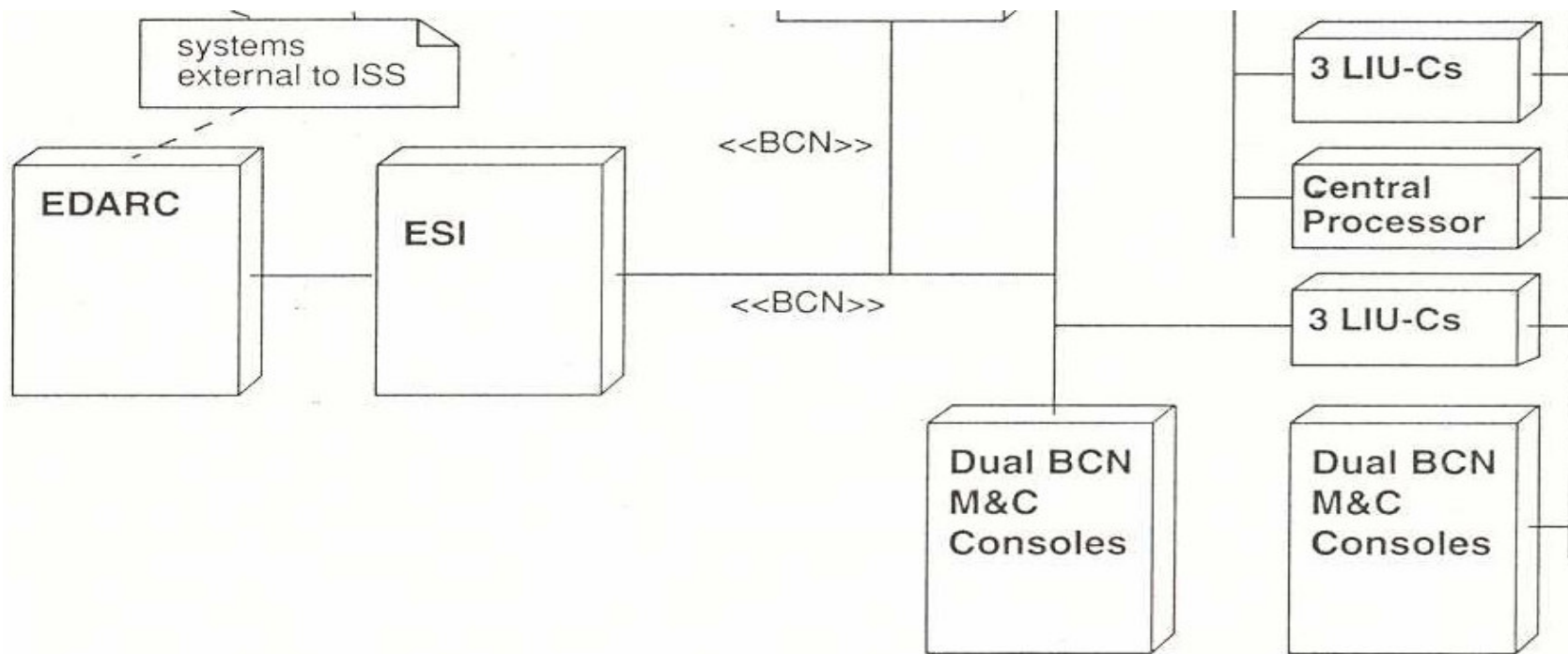


ISSS Physical View (top portion fig 6.5)





ISS Physical View (rest of the figure)





Physical View Notes

- Major elements
- HCS A – Host computer System A (primary)
 - Processes radar and flight-plan info.
 - Output to consoles (radar) and flight-strip printers (flight-plans)
- HCS B – backup Host
- Common Consoles – the workstations
- Local Communications Network – Consoles $\leftrightarrow$ Hosts
 - Diagram flaky here hosts on wrong side
 - Each host has two interface units called LIU-H
 - LCN composed of 4 parallel token ring networks
 1. One supports broadcast of radar info
 2. One for point-to-point between workstations
 3. One provides for recording data for later playback
 4. A spare



Physical View Notes

- Backup Communication Network (BCN) is an ethernet using TCP/IP
- Both LCN and BCN have monitor and control consoles
- Enhance Direct Access Radar Channel (EDARC) provides backup display of info in case of loss of Host
- EDARC supplies raw data to the External System Interface (EIS) processor
- Central processors mainframes that provided record and playback functions for early version of ISSS
- Testing and training subsystem – allow training of new personnel and testing of new equipment without interfering



Module Decomposition View

- Elements called Computer Software Configuration Items (CSCIs) as required by the government software development standard required by the customer
- 5 CSCIs:
 1. Display Management
 2. Common Systems Services
 - General ATC utilities; remember bigger picture ISSS 1/3 of AAS
 3. Recording, analysis and playback
 4. National Airspace System Modification
 - Modifying software on host
 5. IBM AIX operating system



Module Decomposition View

- The CSCIs formed deliverable units software and documentation)
- Tactics:
- Semantic coherence – main one guiding the decomposition
- Abstract common services –
- Record/playback tactic
- Generalizing module – well designed interfaces

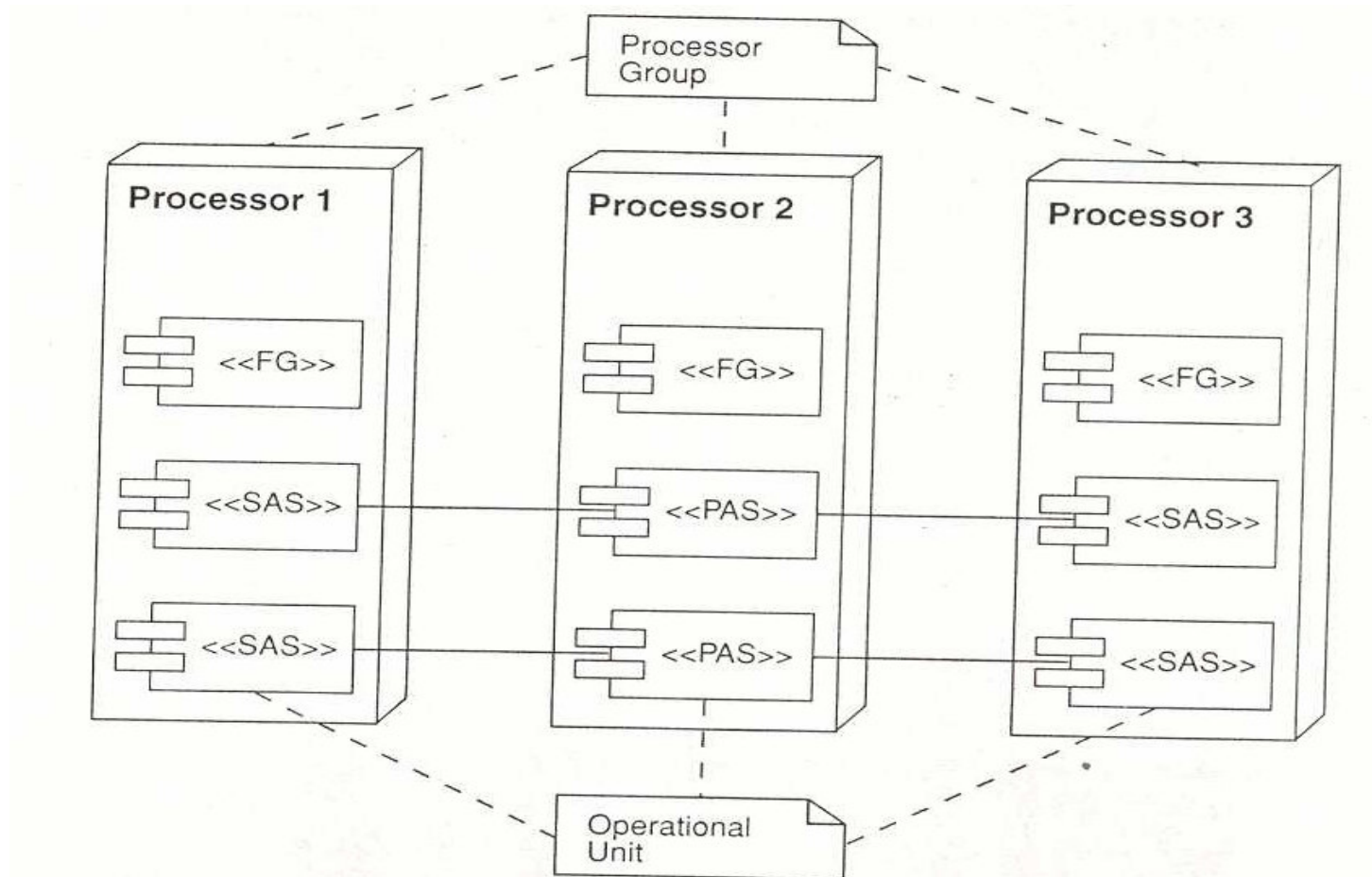


Process View

- Concurrency resides in “applications” roughly processes in Dijkstra’s CSP
 - Ada Main unit – a process schedulable by OS
- ISSS designed to work on more than one processor
 - Processors grouped into “processor groups”
 - Critical to fault tolerance and thus availability
 - One primary the rest backup
 - PAS – primary address space
 - SAS – standby address space
 - Operational unit – the collection of primary and its standbys
 - Function groups are the components not implemented in this fault tolerant fashion (replicated on several groups)



ISSS Functional Groups, Operational Units, Processor Groups and Address Spaces





Primary Failure Switchover

1. PAS fails
2. A standby system SAS is promoted to PAS
3. The new PAS sends messages notifying of the failure and starts providing all services
4. A new SAS is started up to replace to old failed PAS.
5. The new SAS sends message to notify the new PAS
6. Adding an new operational unit is similar but more complex p 140-141

Adding a new Operational Unit



1. Identify necessary input data and its location.
2. Identify where (which Oper Unit / FG) to send output
3. Fit operational unit's communication patterns into system wide acyclic graph such that it remains acyclic and deadlocks will not occur.
4. Design messages to achieve this.
5. Identify internal state data that must be used for checkpointing. (must be included in PAS → SASs)
6. Define messages: message types, data
7. Plan for switchover on failure; test for consistency
8. Ensure processing steps less than a heartbeat
9. Plan data-sharing and synchronization with other Operational Units
10. Not for the faint-hearted(novices) but Code Templates!



Client-Server View

- Communication between PAS elements within operational units (client and server)
- Figure 6.7
- PAS → PAS
- Then each PAS sends updates to its SASs
- The client sends a “service request message”
- The server acknowledges and responds with results
- Within operational units PASes send updated state to SASes
- Within FGs nothing extra just ACK and results



Code View

- Code view – describes how functionality is mapped into code units
- ISSS Code view
- Ada main program
- Subprograms grouped into packages (separately compilable)
- Ada program consists of one or more tasks (threads)
- Applications decomposed into Ada packages

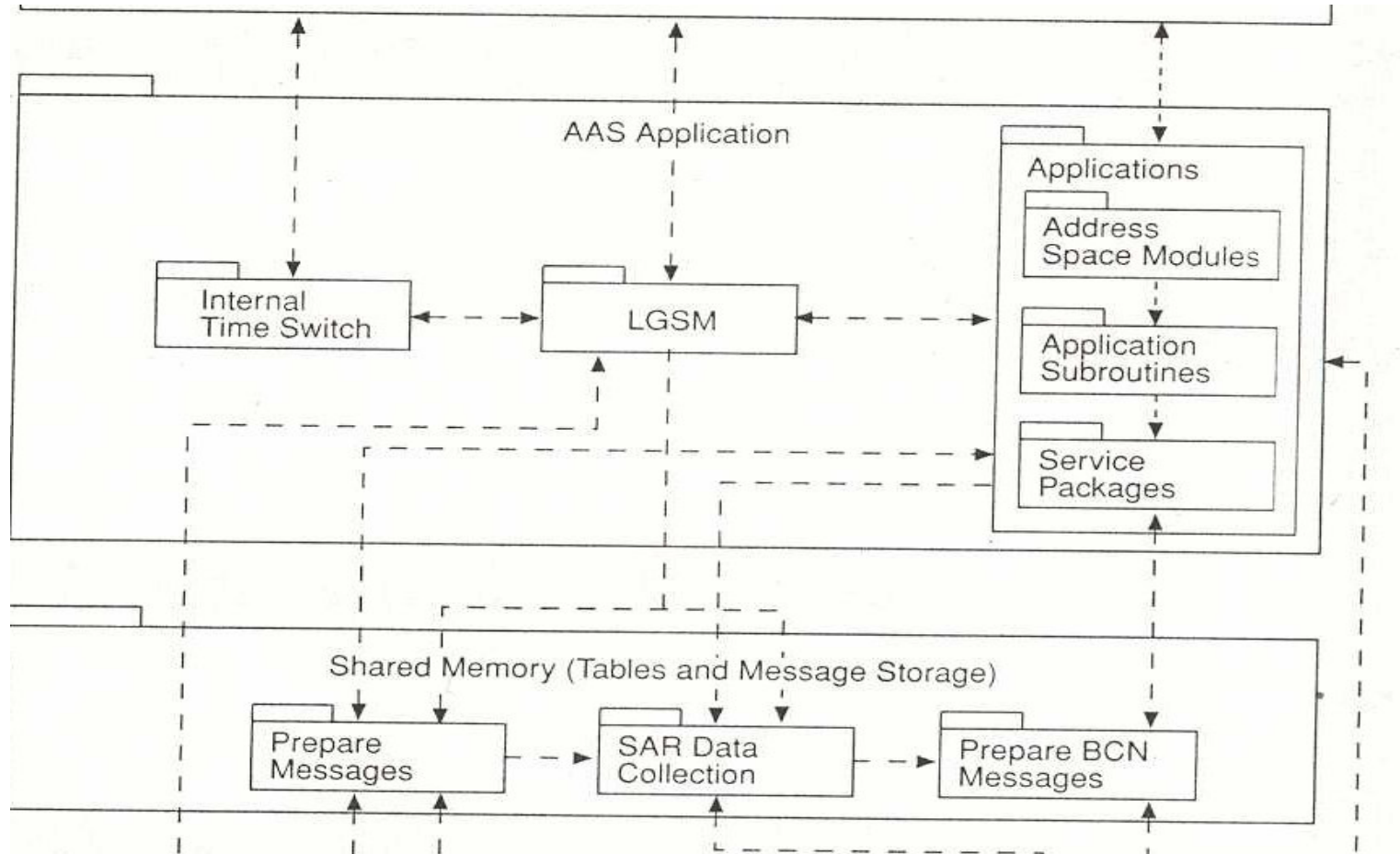


Layered View

- Underlying Operating System, AIX (IBM's version of Unix)
- Layers
- Shared memory (Tables and Message Storage)
- AAS application
- Shared Memory (Tables and Message Storage)
- CAS AIX Kernel Extension
- AIX Kernel

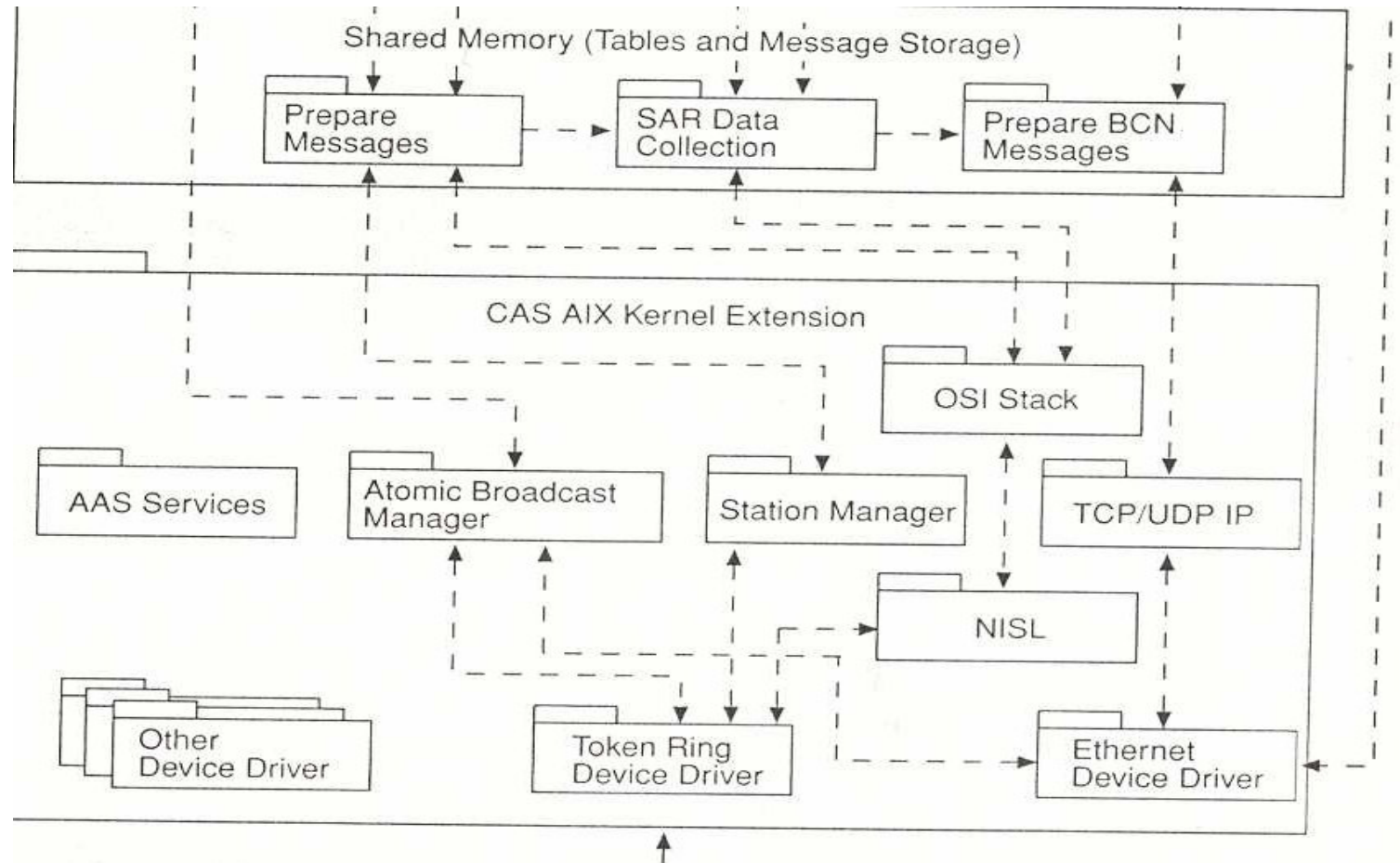


AAS Application Layer





CAS AIX Kernel Extension Layer





Notes on the Layered View

- AIX (unix) in particular does not support fault-tolerant features necessary for ISSS
- Kernel extension
 - Lowest two rows: token ring, ethernet and other device drivers run in kernel address space (supervisor mode)
 - Written in C; must be small trusted reflecting “limit exposure” tactic
 - Atomic Broadcast Manager (ABM) -
 - Station Manager – provides datagram services on LCN
 - NISL – network interface sublayer – provides point to point
 - Local availability Manager manages the availability of suite functions



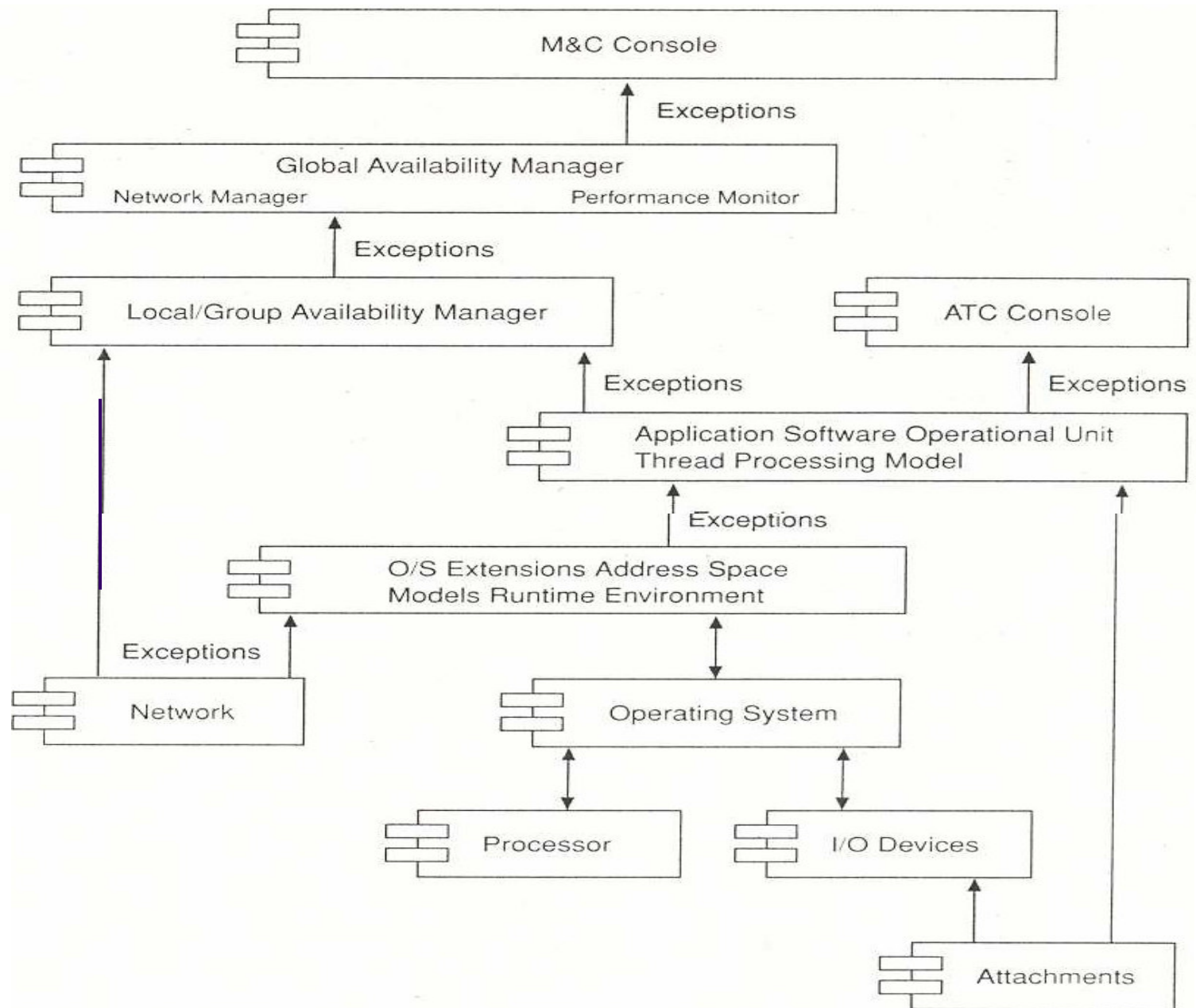
Notes on the Layered View

- Next level up runs outside kernel space
 - Cannot damage AIX
 - Therefore written in Ada to conform to Specifications
 - Prepare messages (prepare BCN messages) – application interface to send/receive LCN messages
 - Local availability Manager keeps track of which process is “primary” so that messages can be sent there
- The Top Layer is where Applications reside
 - Local availability Manager is at this level
 - Responsible for initiation, termination and access to applications
 - Communicates with the LCMs of other console groups
 - Also with Global Availability Management of the M&C consoles
 - Internal Time Synchronization synchronizes the clocks



New views

- There is no exhaustive list! Others possibly helpful.
- Increasing emphasis on achieving quality attributes
 - ➔ development of views addressing quality attribute
- Runtime qualities: the corresponding view is typically a component-and-connector type showing runtime interactions
- For non-runtime qualities (e.g. modifiability) - the view is typically a “module decomposition” type showing how the modules achieve the quality



**Fault
Tolerance
View
ISSS component-and-
connector view**

Notes on the Fault Tolerance View



- Runtime quality → component-and-connector type
- Components of the Fault tolerant hierarchy
 - M&C console
 - Global Availability Manager
 - Local/Group Availability Manager
 - ATC console
 - Application Software Operational Unit (Thread Processing Model)
 - OS extensions Address Space Models
 - Network
 - Operating System
 - Processor
 - I/O devices
- PAS/SAS designed to provide fault-tolerance within single application traps and recovers from errors
- The hierarchy provides for errors that occur cross-application
 - Detecting, isolating and recovering from errors that occur interactions

Notes on the Fault Tolerance Hierarchy



- Each level of the hierarchy
 - Detects errors in itself, peers and all lower levels
 - Handles exceptions from lower levels
 - Diagnoses, recovers, reports or raises exceptions
- Levels from Top to Bottom
 - System monitor and control
 - Global availability
 - Group availability
 - Local availability
 - Application
 - Runtime environment
 - Operating System
 - Physical level: processors, networks, devices

Notes on the Fault Tolerance Hierarchy



- Fault Detection at each level by
 - built-in tests
 - Event time-outs
 - Network circuit tests
 - Group membership protocols
 - Human reaction to alarms
- Fault recovery can be automatic or manual
 - For availability managers recovery is table driven
 - In a PAS there are 4 types of recovery
 1. In a switchover the SAS takes over for the old PAS
 2. A warm restart uses checkpoint data saved to non-volatile memory
 3. Cold restart uses default start-up data
 4. A cutover is used to transition to new logic or data

Notes on the Fault Tolerance Hierarchy



- Fault tolerance of the hardware is done via redundancy
- LCN, BCN, various bridges
- Backup radar and separate channel for it
- Processor hardware replicated within processor group
- Tactics added here –
- component availability used for fault tolerance
 - “Ping/echo”
 - “Heartbeat”
 - “Exception” to transfer errors to the correct place
 - “spare” to perform recovery

Relating the Views



- Additional insight is provided by examining relationships between views
- Mapping one view to another
- In ISSS
- CSCIs are the elements in the module decomposition view (composed of applications)
- Applications (processes) are the elements in the process view and in the client-server view
- Applications are implemented in Ada packages and programs elements of the Code view
- Applications are turned into threads at runtime elements of the concurrency view
- The special quality attribute view (fault-tolerance) uses elements from the process, layer and module views

“Configuration Files” Tactic



- ISSS makes extensive use of the modifiability tactic “configuration files”
- It calls this adaptation data.
- Site-specific data allows configuration of ISSS for each of the 22 en route centers
- This configuration is fairly extensive and powerful
 - E.g., splitting an ATC console window into two
 - “generalize the module” tactic
- Negative side
 - It takes powerful interpretation mechanism to support this level of adaptability at run-time
 - It therefore is complex to maintain the mechanism if changes are required there.
 - Different configurations substantially complicates testing.



“Abstract Common Services” Tactic

- PAS and SAS really comes from the same source
 - No difference in the code
 - Just dynamic state boolean variable “primaryStatus”
 - Code Template Structure (fig 6.10) for all operation units
 - “Abstracting Common Services” tactic
 - Common part is abstracted to template

Code structure Template for Operational Units (providing fault tolerance)



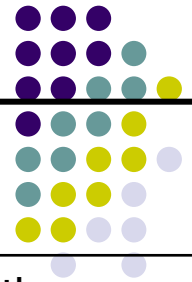
- Initialize();
- Ask for current state
- Loop until terminate == TRUE
- get_event
- case EventType is
- normal - - only for primary (PAS)
- when Send to Process X → send to SASs as well
- when terminate-directive → clean-up; terminate =
- TRUE
- when State-update → update state variables
- (SAS)
- when switch-directive → notify service packages
- of change
- when reconstituteFrom → reconstitute
- when others → log error
- End loop



Code Template affects other Tactics

- Other modifiability tactics addressed by code template
- “anticipation of expected changes”
- “Semantic coherence”
- “generalizing the module”
- Making interfaces part of the template “maintain interface stability” and “adherence to defined protocols”

How ATC Achieves Quality Goals



Goal	How Achieved	Tactic(s) Used
High Availability	Hardware redundancy, software: layered fault detection and recovery	State resynchronization, shadowing, active redundancy, ping, heartbeat, exception, spare
High Performance	Distributed multiprocessors, scheduling and network analysis	Introduce concurrency
Openness	Interface wrapping and layering	Abstract common services, maintain interface stability
Modifiability	Templates and table-driven adaption data; careful assignment of functionality; strict interfaces	Abstract common services, semantic coherence, configuration files, defined protocols, ...
Ability to field subsets	Appropriate separation of concerns	Abstract common services
Interoperability	Client-server division of functionanlity	Adherence to defined protocols, interface stability



ISSS Summary

- Architectural solutions can be the key to achieving the needs of an application (especially quality attribute requirements)
- ISSS
- High availability → fault tolerance
- Longevity → high modifiability, interoperability
- Audit of ISSS before abandoning